Overrun-Triggered Load Shedding on a Nonpreemptive Fixed-Priority Executor: An Empirical Study Building on Exceedance Analysis

Mohamed Ghoneim
Systems and Control Engineering, King Fahd University of Petroleum and Minerals (KFUPM)
Dhahran, Saudi Arabia
g202522450@kfupm.edu.sa

October 2026

Abstract

Execution-time estimates on commodity hardware are estimates, not bounds, so a real-time system must decide at run time what to do when jobs overrun. Recent *exceedance analysis* bounds response times under a bounded amount of overrun and defines a *tolerable exceedance threshold* (TET); its authors suggest that the TET can indicate when a degraded mode should be triggered, but leave runtime mitigation to future work. We study this suggestion empirically. In a discrete-event simulation of a single-core, nonpreemptive, fixed-priority polling executor with a simulated 50 Hz motor-speed control loop next to necessary and optional callbacks, we compare four causal triggers for shedding optional work under correlated execution-time slowdowns: a completed-action busy-time proxy and an exact sliding-window version of CPU utilisation, the control task's response time, and overrun accumulated relative to nominal execution times. We select one configuration per monitor on 16 training seeds for a target of 0.85 optional work retained and evaluate it on 40 disjoint test seeds. The overrun monitor misses 0.14% of the control deadlines, against 0.23% and 0.24% for the two utilisation monitors and 0.22% for the response-time monitor (0.62% without mitigation), while retaining 0.84 of the optional work against 0.84, 0.85 and 0.77; an exploratory frontier comparison gives miss-rate ratios between 0.42 and 0.60. Smaller overrun budgets lower the miss rate at a cost in retained optional work, and the most sensitive budgets become costly under heavier OS noise. We separate the loss of the analysis's deadline certificate from a miss that is already forced by nonpreemptive blocking; in the unmitigated runs 96% of the missed control jobs arrived while an optional callback was executing. We check our implementation of the analysis against the published workload and randomised adversarial simulations (no violation of the bound in 4,200 runs). This is a simulation study with illustrative parameters and provides no guarantee for the mitigated system. Code, seeds and data are provided.

Keywords: real-time systems, exceedance analysis, overload management, load shedding, mixed criticality, nonpreemptive scheduling, control.

1 Introduction

Execution times of real-time software on commodity hardware are routinely estimated from measurements rather than bounded by static analysis [22, 27]. Estimates can be exceeded, for instance through contention on shared resources, thermal throttling or interrupts. *Exceedance analysis* [29]

asks the complementary design-time question: if jobs may exceed their estimated execution times by a bounded total amount within a busy window, are deadlines still met? Its mechanised instantiation for a polling-based, nonpreemptive, fixed-priority scheduler (Rössl) yields, for every callback, a *tolerable exceedance threshold* (TET) $\hat{e}$: the largest total overrun for which the analysis still establishes the callback's deadline under its assumptions [5].

A design-time threshold is most valuable if it can also guide what the running system does. The authors of [5] observe that exceedance metrics can inform runtime mechanisms, for example by indicating when a fail-operational degraded mode may have to be triggered, and leave this integration to future work. This report takes a first empirical step. It asks what a runtime monitor that sheds optional work should observe, and how its threshold relates to the TET.

Research questions. **RQ1** What trade-off between control deadline misses and retained optional work do an overrun-based, a utilisation-based and a response-time-based shedding monitor achieve in a simulated workload? **RQ2** How does this trade-off depend on the overrun budget, expressed as a fraction of the TET? **RQ3** What does nonpreemption imply for reacting to an overrun, and what was executing when the missed control jobs arrived? **RQ4** How sensitive are the answers to disturbance severity and to the OS-noise level?

Contributions.

1. An executable simulation model of a Rössl-style executor with a co-simulated control plant, and a Python reimplementation of the exceedance-analysis equations. With scheduler overheads omitted in the comparison with the published workload, the computed TETs differ from the published values by less than 0.1 ms; no bound violation was observed in 4,200 randomised adversarial checks (Section 5).
2. A comparison of four causal monitor implementations, each configured on training seeds and evaluated on disjoint test seeds (Section 6). At the 0.85 training target the selected overrun monitor has about 40% fewer misses than the utilisation proxy, 43% fewer than the exact-window utilisation monitor and 37% fewer than the response-time monitor, with test optional-work fractions of 0.844, 0.837, 0.852 and 0.774 respectively; an exploratory frontier comparison is reported separately.
3. A distinction between the loss of the analysis's sufficient deadline certificate and a deadline miss that is already forced by an executing nonpreemptive callback (Observation 1), together with a classification showing that 96% of the unmitigated missed control jobs arrived while an optional job was executing.
4. Sensitivity results for the tested workload: smaller overrun budgets lower the miss rate at a cost in retained optional work, and the most sensitive budgets cost more under higher OS-noise rates. Exploratory wider-grid, higher-peak-slowdown and exact-window-utilisation experiments preserve the ordering of the miss rates at the evaluated configurations, whose retained optional work is not generally matched.

What this report is not. Switching to a degraded mode when an execution-time budget is overrun is classical in mixed-criticality scheduling and in automotive operating systems (Section 8); we do not claim the idea. We do not prove any guarantee for the mitigated system: the theorem of [5] covers the unmitigated scheduler, and a run-time mode change is outside it. All evidence is from simulation with illustrative parameters.

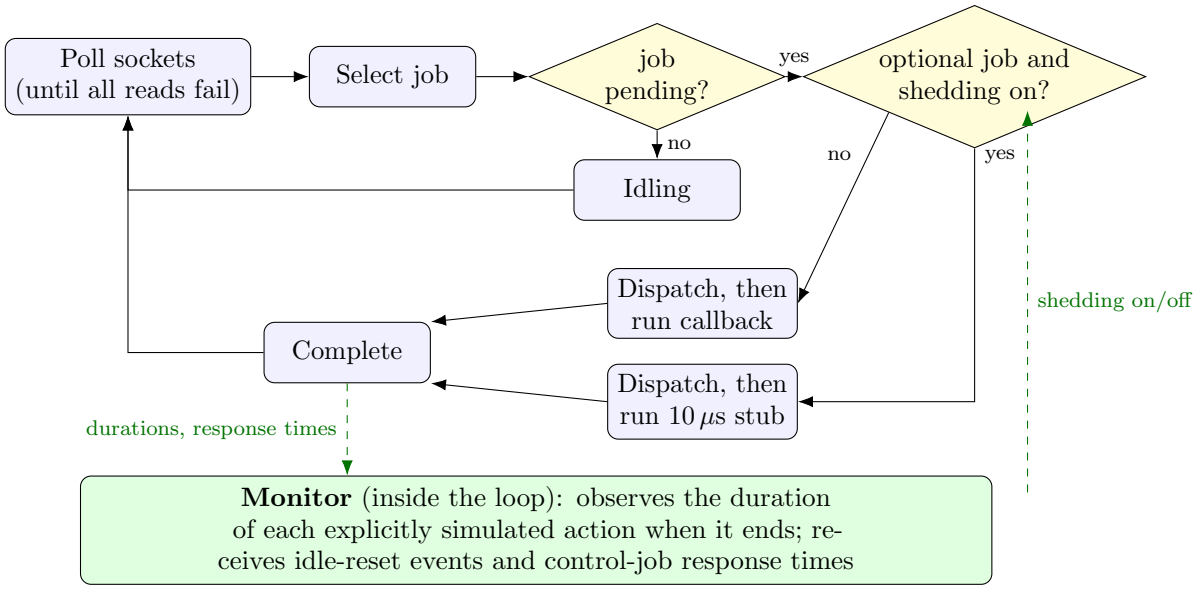


Figure 1: Executor control flow and monitor hooks. The monitor observes an action when it ends and is consulted once per optional job, immediately before the dispatch action. That decision fixes whether the callback body or the stub runs after dispatch; a running callback is never interrupted.

2 Background

2.1 A polling-based nonpreemptive executor

We model the scheduler loop of [5] as we read it. The scheduler serves s sockets, one per callback. One iteration consists of a *polling phase* (non-blocking reads on the sockets in round-robin order until a read has failed on every socket), a *selection* action that picks the highest-priority pending job (FIFO within a callback), and then either an *idling* action if nothing is pending, or *dispatch*, execution of the callback *to completion*, and a *completion* action. A message is readable at the start of a read if it arrived at or before that instant. The response time of a job is the time from message arrival to the completion action. Figure 1 shows the loop with the hooks used by our monitors.

2.2 Exceedance analysis and the tolerable exceedance threshold

Let NET_i be the nominal execution time of callback χ_i (typically a high percentile of measurements) and $\varepsilon = 1\,\mu$s the smallest time unit. We restate the analysis of [5] in the form we implemented. Scheduler overheads are folded into an inflated cost

$$C_{\mathrm{ovh}}(\chi_i) = s\,\mathrm{FR} + \mathrm{Sel} + \mathrm{Disp} + \mathrm{NET}_i + \mathrm{Compl} + (s-1)\,\mathrm{FR}, \tag{1}$$

where FR, Sel, Disp and Compl are the NETs of a failed read, selection, dispatch and completion, and two pseudo-tasks model polling (τ_{read}, highest priority, arrival curve the sum of all callbacks') and idling (τ_{idle}, lowest priority). Let $\mathrm{hep}(\tau)$ and $\mathrm{lp}(\tau)$ denote the tasks with higher-or-equal and lower priority, $\alpha(\tau', \Delta)$ the arrival curve (here $\lceil \Delta/T \rceil$ for period T), and

$$B(\tau) = \max_{\tau' \in \mathrm{lp}(\tau)} \big(C_{\mathrm{ovh}}(\tau') - \varepsilon\big), \qquad H(\tau, \Delta) = \sum_{\tau' \in \mathrm{hep}(\tau)} C_{\mathrm{ovh}}(\tau')\,\alpha(\tau', \Delta). \tag{2}$$

Callback	Period = deadline	NET	Priority	Role
`ctrl`	20 ms	1.5 ms	1 (highest)	simulated PI motor-speed loop (necessary)
`sens`	40 ms	4 ms	2	synthetic load: sensor processing (necessary)
`tele`	50 ms	8 ms	3	synthetic load: telemetry (optional)
`viz`	100 ms	12 ms	4 (lowest)	synthetic load: visualisation (optional)

Table 1: The simulated workload. Priorities are fixed; arrivals are periodic with a random phase per seed.

If the jobs in a busy window exceed their inflated NETs by a total of e, the busy window is bounded by the least $L(e) > 0$ with $L(e) \geq e + B(\tau) + H(\tau, L(e))$, and the response time of τ by

$$R(e) = \max_{A \in SP(e)} \big(F_A(e) + C_{\text{ovh}}(\tau) - \varepsilon - A\big), \tag{3}$$

$$F_A(e) = \min\{F > A : F \geq \varepsilon + B(\tau) + H(\tau, F) - C_{\text{ovh}}(\tau) + e\}, \tag{4}$$

with $SP(e) = \{A < L(e) \mid \alpha(\tau, A) \neq \alpha(\tau, A + \varepsilon)\}$. The *tolerable exceedance threshold* is $\hat{e}(\tau) = \max\{e \mid R(e) \leq D\}$ (Corollary 1 of [5]). Two derived quantities are used later: the *recovery time* $L(\hat{e})$ of the lowest-priority callback, and the *necessary slowdown* $\hat{s} = 1 + \hat{e}/\sum_j C_{\text{ovh}}(\tau(j))$ over the jobs that can interfere in the busy window plus the largest potential blocker. We implement the analysis in integer microseconds. Our implementation carries none of the machine-checked guarantees of the original; Section 5 describes how we validated it.

3 System, Disturbances and Monitors

3.1 Workload and control plant

The simulated workload runs on one core (Table 1). The highest-priority callback `ctrl` drives a simulated 50 Hz PI speed loop. The other three callbacks are synthetic execution demands named after the roles they stand for: `sens` is necessary sensor processing, `tele` (telemetry) and `viz` (visualisation) are optional; they do not implement those functions. Basic-action NETs (in μs) are FR = 8, successful read = 25, Sel = 10, Disp = 10, Compl = 10, Idling = 20, and the number of sockets is $s = 4$. These values are illustrative and are not measurements of a particular platform. Nominal callback utilisation is 45.5%.

The plant is a first-order motor, $\tau_m \dot{\omega} = -\omega + u - d(t)$ with $\tau_m = 0.1$ s, a load torque $d(t)$ of 0.15 applied for 2.3 s every 5.1 s, and a reference that changes to a new level drawn from $[0.2, 0.9]$ every 0.5 s (the same sequence in every run). The discrete PI controller ($K_P = 2$, $K_I = 20$, sample time 20 ms, saturation ± 1.5, conditional-integration anti-windup) samples the speed when the control message *arrives* (a hardware latch) and applies its output when the control callback *completes*, so tracking depends on response time. The gains are illustrative: a constant latency of 40 ms doubles the RMS tracking error of this loop and 80 ms increases it about eightfold (`calibration_latency.py` in the artefact reproduces this), so a few missed periods matter. The tracking-error integral is evaluated in closed form between events.

3.2 Execution times and disturbances

In the stochastic mode, a job executes for $\text{NET}_i \cdot U(0.6, 1)$ and a basic action for $0.8 \cdot \text{NET}$, so nominal jobs mostly lie *below* their NET. Two disturbance mechanisms act on time only, so they are identical for every policy: *OS noise* (stolen time from 30 events/s with log-normal duration,

Scenario	Windows	Constant / peak slowdown	Profile	Mean gap
nominal	none (OS noise only)	–	–	–
spike	10–40 ms	3–8	constant	1.2 s
burst	0.3–0.8 s	1.2–3	constant	3 s
ramp	1.5–3 s	peak 1.3–3	triangular (rise 70%, fall 30%)	4 s
burst, high peak	as burst	2–3.5	constant	3 s
ramp, high peak	as ramp	peak 2–3.5	triangular	4 s

Table 2: Disturbance scenarios. Window durations and the constant slowdown factor (spike, burst) or the peak factor (ramp) are drawn uniformly from the stated ranges. In the ramp scenarios the factor rises from 1 to the peak and returns to 1, so it is low at the edges of a window. Windows are separated by at least 0.2 s plus an exponential gap. (In the artefact the high-peak variants are named `burst_harsh` and `ramp_harsh`.)

median 60 μs, capped at 1 ms, extending whichever action is running), present in every scenario; and *slowdown windows*, during which every action that *starts* inside the window takes $s(t) \geq 1$ times longer. Table 2 defines the scenarios. The window rates and severities were chosen in exploratory runs on training seeds so that the unmitigated system misses roughly 0.5–1% of its control deadlines in each disturbed scenario (Table 9). The slowdown ranges include mild factors that the nominal headroom absorbs only partly: because nominal job times are spread over 0.6–1.0 of their NET, a factor below about 1.25× often causes no overrun, but can still cause one for a job whose nominal time is close to its NET. Two higher-peak variants are used in a robustness check.

3.3 Monitors

All monitors run inside the scheduler loop (as they would in a single-threaded executor) and are *causal*: they see the duration of each action when it ends and are consulted once per optional job, immediately before its dispatch. While shedding is on, the callback body of an optional job is replaced by a 10 μs stub (the job counts as shed). Each monitor adds 3 μs to every dispatch. Once a monitor triggers, shedding stays on for a *hold time* H and is extended by every further trigger. The monitors differ only in the trigger:

Utilisation proxy (θ, W). At the end of each busy action, sum the full durations of the busy actions that *ended* in the last W and trigger when the sum exceeds θW. Busy actions are successful reads, selections that found a job, dispatches, callback bodies or discard stubs, and completions; failed reads, empty selections and idling are excluded. An action that straddles the left edge of the window counts with its full duration, so this is a completed-action approximation of CPU utilisation and its value can exceed one. It is the utilisation baseline of the first analysis.

Utilisation, exact window (θ, W). The same trigger with exact sliding-window accounting: the part of an action that lies outside the window is clipped, so the value is a true busy fraction in $[0, 1]$. It is a stricter baseline alongside the proxy.

Response time (φ). Triggers when a completed control job has a response time above $\varphi \cdot D_{\texttt{ctrl}}$, i.e. it reacts to a symptom at the protected task.

Overrun budget (γ). Accumulates $\sum_a \max(0, d_a - \mathrm{NET}_a)$ over the explicitly simulated actions and callback bodies a completed since the most recent *idle reset*, and triggers when the sum reaches $\gamma\, \hat{e}_{\texttt{ctrl}}$, where $\hat{e}_{\texttt{ctrl}} = 6.005$ ms is the analytic TET of `ctrl` for the workload with all callbacks enabled. The reset occurs at the end of an idling action that followed a selection that found no pending job; messages may arrive during that idling action, so this is an implementation heuristic and not the quiet instant of the formal analysis. The per-action accumulator is also not the whole-job exceedance of the analysis [5], and we do not prove a

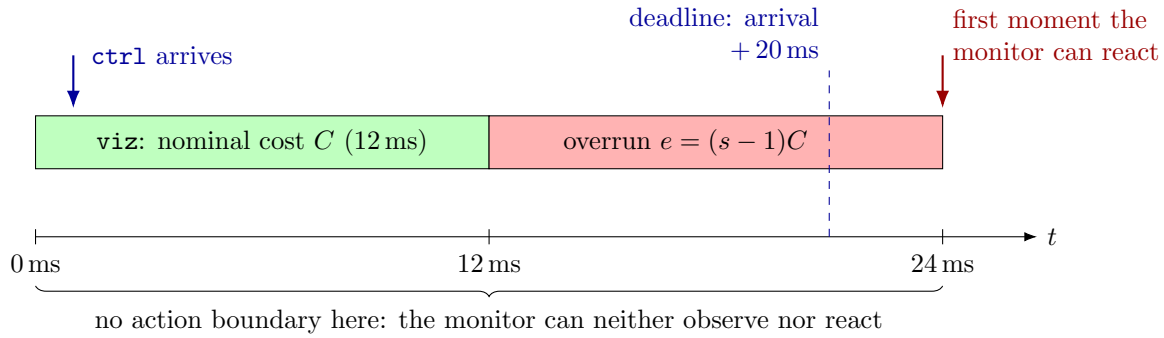


Figure 2: A forced miss (schematic). `viz` is dispatched just before `ctrl` arrives and is slowed by a factor $s = 2$. The control job cannot start before the callback ends, which is after its deadline, so it misses whatever the monitor does afterwards.

correspondence between the two: it is a heuristic, not a sound online bound on the exceedance e.

Two diagnostic reference policies are included. *Always shed* replaces every optional callback body by the stub; its measured miss rate is a reference result, not a proved lower bound. The *oracle* sheds an optional job if its sampled undisturbed cost times the slowdown factor in force at the decision exceeds its NET; it uses information unavailable to the causal monitors, does not predict OS-noise delay or a change of slowdown between the decision and the start of the callback body, and is not an optimality bound.

3.4 Limits of completion-based monitoring

Figure 2 illustrates the difficulty: a nonpreemptive callback that has started cannot be stopped, so overrun incurred inside it cannot be observed, let alone acted on, before it ends.

Observation 1 (Certificate loss and nonpreemptive blocking)**.** *Let τ be a protected callback with deadline D and $\hat{e} = \max\{e : R(e) \le D\}$ its tolerable exceedance threshold.* (a) Certificate. *For an exceedance budget $e > \hat{e}$ the sufficient test of the analysis does not certify the deadline of τ. If all actions of one job of a callback χ are slowed by a factor $s \ge 1$, the exceedance of that job is $(s-1)C_{\mathrm{ovh}}(\chi)$, which exceeds $\hat{e}$ if and only if*

$$s > s^{\star}(\chi) = 1 + \frac{\hat{e}}{C_{\mathrm{ovh}}(\chi)}. \tag{5}$$

(b) Forced miss. *Suppose a job of τ is released at time a while a nonpreemptive callback body is executing, and let f be the completion time of that body. The job of τ cannot start before f, so if $f > a + D$ it misses its deadline, and a monitor that acts only at action boundaries cannot prevent this particular miss. For a callback with nominal body time NET_{χ} slowed by s and a release just after its dispatch, this holds whenever $s\,\mathrm{NET}_{\chi} > D$.*

The two statements are different. Statement (a) concerns the guarantee of an upper bound and is an algebraic threshold; it is neither necessary nor sufficient for a miss, because $R(e)$ is an upper bound. Statement (b) is a concrete, sufficient condition for a miss; it ignores the execution time of τ itself and is therefore conservative. Neither is a claim about every possible monitor design (for instance one that can interrupt callbacks) or about what the evaluated monitors achieve. Table 3 gives both thresholds for our workload; for `viz` they are 1.50 and 1.67.

Callback	Kind	C_{ovh} (ms)	NET (ms)	$s^\star$ (certificate lost)	D/NET (miss forced)
`sens`	necessary	4.09	4.0	2.47	5.00
`tele`	optional	8.09	8.0	1.74	2.50
`viz`	optional	12.09	12.0	1.50	1.67

Table 3: Thresholds for the callbacks that can block `ctrl`: the algebraic factor $s^\star = 1 + \hat{e}_{\texttt{ctrl}}/C_{\text{ovh}}$ at which one job's exceedance exceeds the TET of `ctrl` (the certificate of the analysis is lost), and the factor $D_{\texttt{ctrl}}$/NET above which a control job released just after the dispatch of that callback is forced to miss its deadline.

4 Experimental Methodology

4.1 Simulator and common random numbers

The simulator explicitly advances polling, selection, dispatch, callback and completion actions in integer microseconds, co-simulates the plant, and invokes the monitor hooks at the points of Figure 1. To keep 40 s runs tractable it fast-forwards whole idle polling iterations while nothing is pending; OS-noise events inside a skipped interval are replayed as stretched idle actions followed by an idle reset; the remaining idle iterations are not simulated one by one, so equivalence with a fully action-by-action simulation is not proved (Section 5). Section 5 quantifies the effect. For a given seed, every policy sees the same message phases, the same per-job execution-time draws, the same OS-noise events and (per scenario) the same slowdown windows: noise depends on the seed only, so the nominal scenario of a seed is the exact counterfactual of its disturbed scenarios. Runs last 40 s of simulated time; job metrics use jobs arriving after the first 0.5 s, and jobs still unfinished 1 s after the end count as misses.

4.2 Protocol and statistics

Metrics. *Miss rate*: fraction of control jobs whose response time exceeds the 20 ms deadline. *Optional work kept*: $\sum_i \text{NET}_i \cdot \#\{\text{executed (not shed) jobs of } \chi_i\} / \sum_i \text{NET}_i \cdot \#\{\text{arrived jobs of } \chi_i\}$ over `tele` and `viz`. *Longest run*: the longest sequence of consecutive control misses in a run, averaged over seeds and scenarios. *RMS excess*: the percentage increase in RMS tracking error over the same seed's nominal run without monitor. *False triggers*: recorded shedding episodes that begin outside every slowdown window (plus 1 s), divided by the nominal run length in minutes; unlike the job metrics the counter does not exclude the warm-up or the final 1 s, so it is a diagnostic count and not a rate over the evaluation interval. Metrics are averaged over the four scenarios with equal weight to give one value per seed ("pooled"); the *seed* is the unit of analysis.

Operating points and comparison. For each monitor family and each target $\tau \in \{0.80, 0.85, 0.90\}$ we take the training-seed configuration with the lowest pooled miss rate among those whose pooled optional work is at least τ, and report its test-seed performance. This is the main comparison. Because the selected points of different families need not keep the same optional work on the test seeds, we also report *exploratory frontiers*: the lower-left Pareto frontier of (optional work, miss rate) over all tested configurations of a family on the test seeds (together with the two anchors no mitigation and always shed), linearly interpolated at τ. An interpolated point need not correspond to an evaluated policy, and because the frontier picks the best test configurations of each family it is optimistic for every family (most for the families with the most configurations). For selected-policy means and paired differences we use 5,000 bootstrap resamples of the 40 test seeds and percentile

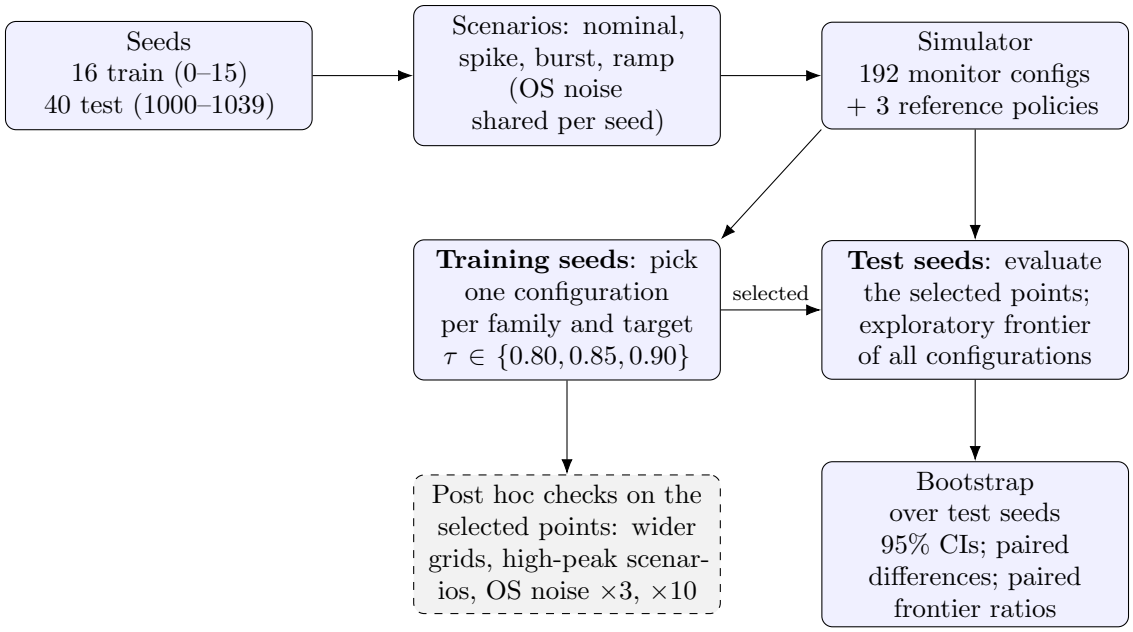


Figure 3: Evaluation protocol. Configurations are chosen on training seeds and evaluated on disjoint test seeds. The frontiers are built from the test results and are exploratory; the response-time and exact-window utilisation baselines and the post hoc checks were added after results had been inspected.

intervals. For the frontiers we use 2,000 resamples and, within a resample, *the same draw of seeds for all families*, recomputing every frontier; the ratio intervals therefore preserve the pairing across families, and the reported ratio is that of the point-estimate frontiers.

Grids. The candidate grids are:

- Utilisation proxy: $\theta \in \{0.50, 0.55, 0.60, 0.70, 0.80, 0.90, 1.00, 1.20\}$, $W \in \{50, 100, 200\}$ ms, $H \in \{100, 300, 1000\}$ ms (72 configurations).
- Utilisation, exact window: $\theta \in \{0.50, 0.55, 0.60, 0.70, 0.80, 0.90, 0.95, 0.98\}$ (a true busy fraction cannot exceed one), with the same W and H (72 configurations).
- Response time: $\varphi \in \{0.25, 0.35, 0.50, 0.65, 0.80, 1.00, 1.25\}$, with the same H (21 configurations).
- Overrun: $\gamma \in \{0.05, 0.10, 0.20, 0.35, 0.50, 0.75, 1.00, 1.50, 2.50\}$, with the same H (27 configurations).

The utilisation families have the largest candidate grids; a larger grid gives more chances to fit the training seeds and does not by itself make the tuning balanced.

5 Validation of the Analysis Implementation

Published workload. The 2026 paper reports TETs and recovery times for a workload of seven matrix-multiplication callbacks. We reproduced them from the published (rounded) periods and NETs, with the scheduler's basic-action NETs set to zero because we did not have measured values; this is a comparison with a simplified reconstruction, not an exact replication of the original parameterisation. Table 4 shows TET differences below 0.1 ms (largest 0.09 ms) for all seven callbacks and recovery-time differences below 0.1 ms for six of them. For the lowest-priority callback our recovery bound is 200.0 ms at the computed TET and 292.94 ms one microsecond above it, against

	TET (ms)		Recovery time (ms)		
Callback	paper	ours	paper	ours	ours at TET+1 µs
`MATRIX_120`	97.85	97.90	352.40	352.30	352.30
`MATRIX_128`	62.39	62.43	198.59	198.54	198.54
`MATRIX_136`	66.36	66.41	295.55	295.46	295.46
`MATRIX_144`	110.95	111.03	365.51	365.43	365.43
`MATRIX_152`	24.39	24.44	160.59	160.55	160.55
`MATRIX_160`	63.80	63.89	293.00	292.94	292.94
`MATRIX_168`	63.80	63.89	292.99	200.00	292.94

Table 4: Reproduction of the published tolerable exceedance thresholds and recovery times (Table III of [5]) with zero scheduler overheads. The last column evaluates the lowest-priority callback's recovery bound one microsecond above the TET of each row.

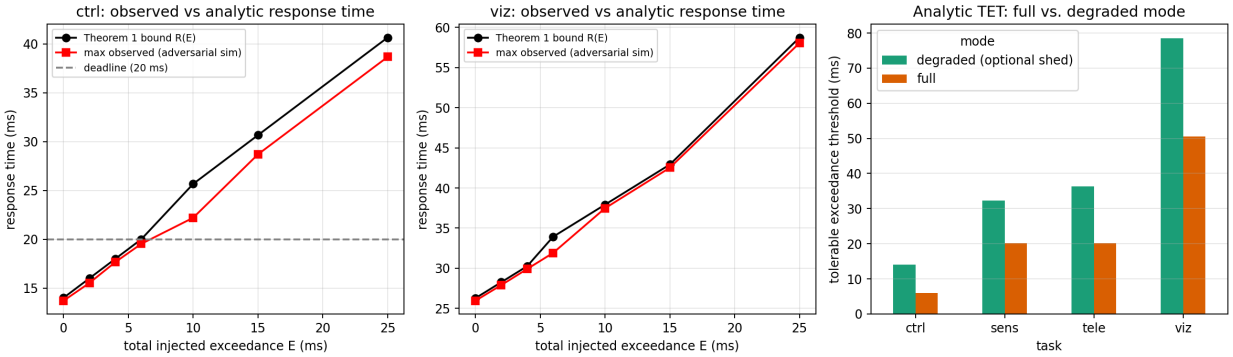


Figure 4: Left, centre: analytic bound $R(E)$ and the maximum response time observed under adversarial exceedance injection, for `ctrl` and `viz`. Right: tolerable exceedance thresholds with all callbacks running and with optional bodies replaced by stubs.

292.99 ms in the paper. This sensitivity is consistent with a discontinuity of the arrival curves at that point, but the comparison alone does not rule out other modelling or implementation differences.

Adversarial simulation. For every callback of our workload and seven values of the total exceedance E (0, 2, 4, 6, 10, 15 and 25 ms) we ran 150 randomised trials with every action at exactly its NET and the exceedance E split over one to three jobs chosen among those executing shortly before or during a random victim job (found in a first pass without injection), with half of the trials using near-synchronous releases. No response time exceeded $R(E)$ in the 4,200 evaluated runs (28 task/exceedance cells); the observed maxima reached between 72% and 99% of the bounds (98–99% at $E = 0$). These finite checks support the consistency of the simulator and the analysis implementation and show close agreement in several cells; they do not prove soundness, model equivalence or tightness. For `ctrl` the largest certifiable integer budget is $\hat{e}_{\texttt{ctrl}} = 6.005$ ms and the bound reaches the 20 ms deadline at $E = 6$ ms (Figure 4, Table 5).

Idle fast-forward. The simulator skips whole idle polling iterations while nothing is pending. OS-noise events that fall inside a skipped interval are replayed as stretched idle actions followed by an idle reset, so the overrun accumulator sees them as it would action by action; the remaining idle iterations are not simulated one by one. To check this approximation we simulated a sample of 96 paired runs (four monitors, four scenarios, six seeds, 10 s of simulated time each) with and

E (ms)	`ctrl`	`sens`	`tele`	`viz`
0	13.70 / 13.99	17.75 / 18.08	25.80 / 26.17	25.94 / 26.25
2	15.52 / 15.99	18.76 / 20.08	27.69 / 29.83	27.89 / 28.25
4	17.66 / 18.00	21.18 / 22.08	27.88 / 31.83	29.90 / 30.25
6	19.53 / 20.00	23.53 / 24.08	33.28 / 33.83	31.91 / 33.92
10	22.20 / 25.66	23.47 / 29.75	34.25 / 37.83	37.47 / 37.92
15	28.69 / 30.66	34.24 / 34.75	42.37 / 42.83	42.52 / 42.92
25	38.67 / 40.66	41.72 / 50.58	47.93 / 66.83	58.07 / 58.75

Table 5: Maximum observed response time / analytic bound $R(E)$ (ms) over 150 adversarial trials per cell.

Policy	Configuration (chosen on train seeds)	Control misses (%)	Optional work kept	Longest run	RMS excess (%)
no mitigation	–	0.624 [0.568, 0.682]	1.000 [1.000, 1.000]	1.21	0.78
utilisation proxy	θ=1.00, W=50 ms, H=1000 ms	0.230 [0.206, 0.255]	0.837 [0.828, 0.845]	1.04	0.10
utilisation, exact window	θ=0.50, W=100 ms, H=300 ms	0.241 [0.215, 0.267]	0.852 [0.847, 0.858]	1.04	0.11
response-time monitor	φ=0.50, H=300 ms	0.220 [0.195, 0.244]	0.774 [0.713, 0.830]	1.04	-0.23
overrun-budget monitor	γ=0.50, H=300 ms	0.138 [0.115, 0.161]	0.844 [0.838, 0.850]	0.69	0.05
oracle (clairvoyant)	–	0.031 [0.022, 0.041]	0.899 [0.894, 0.904]	0.19	0.08
always shed	–	0.027 [0.018, 0.037]	0.000 [0.000, 0.000]	0.19	-1.89

Table 6: Test-seed results (40 seeds, pooled over four scenarios; mean and 95% bootstrap CI) of the configurations chosen on training seeds for a target of 0.85 optional work. "Longest run" is the mean over seeds and scenarios of the longest run of consecutive misses; RMS excess is relative to the same seed's nominal run.

without the skip. Control miss rates were identical in 94.8% of the pairs, with a largest difference of 0.42 percentage points (about two control jobs), and mean miss rates of 0.355% with the skip and 0.355% without it. Optional work was identical in 90.6% of the pairs, with a largest difference of 0.035; for the overrun monitor the mean retained optional work was 0.831 with the skip and 0.830 without it. This is a finite check on a sample, not a proof of equivalence.

6 Results

6.1 Headline comparison

Table 6 gives the test-seed results of the configurations selected on the training seeds for a target of 0.85 optional work. Without mitigation the system misses 0.62% of its control deadlines (pooled over the four scenarios). The selected overrun monitor ($\gamma = 0.50$, $H = 300\,\text{ms}$) misses 0.14%, against 0.23% for the utilisation proxy, 0.24% for the exact-window utilisation monitor and 0.22% for the response-time monitor. It retains 0.844 of the optional work, against 0.837, 0.852 and 0.774 respectively; its miss rate is the lowest of the four, and its retention lies between the two utilisation monitors and the response-time monitor. The response-time configuration kept 0.868 of the optional work on the training seeds but 0.774 on the test seeds, so its comparison is not at matched retention. The oracle reaches 0.03% at 0.899 optional work; always shedding reaches 0.03% with no optional work. Both are diagnostic references, not bounds for other policies.

The paired test-seed differences of these selected configurations (Table 7) are, for overrun minus utilisation proxy, $-0.092\,\text{pp}$ (95% CI -0.105 to -0.078) in misses with $+0.008$ optional work; for overrun minus exact-window utilisation, $-0.103\,\text{pp}$ (95% CI -0.114 to -0.091) with -0.008 optional work (the exact-window monitor retains slightly more); and for overrun minus response time, $-0.082\,\text{pp}$ (95% CI -0.098 to -0.064) with $+0.071$ optional work. The response-time monitor's

Comparison	Target	Δ misses (pp)	Δ optional work
overrun − util. proxy	0.80	-0.120 [-0.136, -0.105]	0.016 [0.009, 0.024]
overrun − util. proxy	0.85	-0.092 [-0.105, -0.078]	0.008 [0.002, 0.014]
overrun − util. proxy	0.90	-0.134 [-0.148, -0.119]	0.008 [0.004, 0.013]
overrun − util. exact	0.80	-0.117 [-0.132, -0.103]	0.019 [0.012, 0.026]
overrun − util. exact	0.85	-0.103 [-0.114, -0.091]	-0.008 [-0.010, -0.006]
overrun − util. exact	0.90	-0.120 [-0.133, -0.107]	0.009 [0.007, 0.012]
overrun − resp.-time	0.80	-0.103 [-0.118, -0.089]	0.014 [0.003, 0.025]
overrun − resp.-time	0.85	-0.082 [-0.098, -0.064]	0.071 [0.016, 0.131]
overrun − resp.-time	0.90	-0.128 [-0.146, -0.111]	-0.002 [-0.009, 0.005]

Table 7: Paired differences (overrun minus baseline) of the configurations selected on training seeds for each target, evaluated on test seeds; pp = percentage points.

Optional work	Util. proxy (%)	Util. exact (%)	Resp.-time (%)	Overrun (%)	Ratio overrun/proxy	Ratio overrun/exact	Ratio overrun/resp.
0.80	0.205 [0.184, 0.226]	0.203 [0.180, 0.225]	0.189 [0.168, 0.210]	0.086 [0.072, 0.101]	0.42 [0.37, 0.47]	0.42 [0.37, 0.48]	0.45 [0.40, 0.51]
0.85	0.239 [0.213, 0.269]	0.232 [0.207, 0.259]	0.222 [0.193, 0.256]	0.134 [0.112, 0.157]	0.56 [0.50, 0.61]	0.58 [0.52, 0.63]	0.60 [0.54, 0.66]
0.90	0.285 [0.251, 0.321]	0.275 [0.245, 0.306]	0.277 [0.246, 0.313]	0.151 [0.125, 0.180]	0.53 [0.48, 0.58]	0.55 [0.50, 0.60]	0.54 [0.49, 0.60]

Table 8: Exploratory interpolation of the empirical test-set frontiers at three optional-work levels, and ratios of the overrun frontier to each baseline frontier with 95% percentile intervals from a bootstrap that shares one draw of seeds across families. Interpolated points are not evaluated policies.

selected configuration kept 0.868 on the training seeds but only 0.774 on the test seeds, so that comparison is not at matched retention. All of these comparisons apply to the selected configurations and their observed retention levels.

Exploratory frontiers. To compare the families at a common retention we also interpolate the empirical Pareto frontier of each family on the test seeds (Table 8, Figure 5). The frontier chooses the best test configuration of each family, so it is optimistic for every family, most for the families with the most configurations, and an interpolated point is not an evaluated policy. With these caveats, at a common retention between 0.80 and 0.90 the miss rate of the overrun monitor is between 0.42 and 0.60 times that of the other monitors (for example 0.56 [0.50, 0.61] against the utilisation proxy, 0.58 [0.52, 0.63] against the exact-window monitor and 0.60 [0.54, 0.66] against the response-time monitor at 0.85), with paired bootstrap intervals below one, i.e. roughly 1.7–2.4 times fewer misses.

6.2 Scenarios and mechanism

Table 9 breaks the selected configurations down by scenario. In the burst scenario the overrun monitor misses 0.025% against 0.972% unmitigated and 0.153%, 0.100% and 0.128% for the proxy, the exact-window monitor and the response-time monitor (retained optional work 0.81 against 0.79, 0.83 and 0.75). In the ramp scenario the values are 0.010% against 0.649% unmitigated and 0.080%, 0.067% and 0.108% (retained 0.78 against 0.84, 0.78 and 0.74). In the spike scenario the overrun monitor misses 0.516% against 0.876% unmitigated and 0.689%, 0.796% and 0.643% (retained 0.79 against 0.72, 0.80 and 0.74), well above the oracle (0.123%). Because retention differs between monitors within a scenario, these per-scenario points are not matched comparisons; the frontier panels of Figure 5 are an exploratory interpolation at common retention.

Figure 6 shows one burst from a single test seed, chosen as the burst window with the most unmitigated misses among the first dozen test seeds (20 s runs). Without mitigation the control

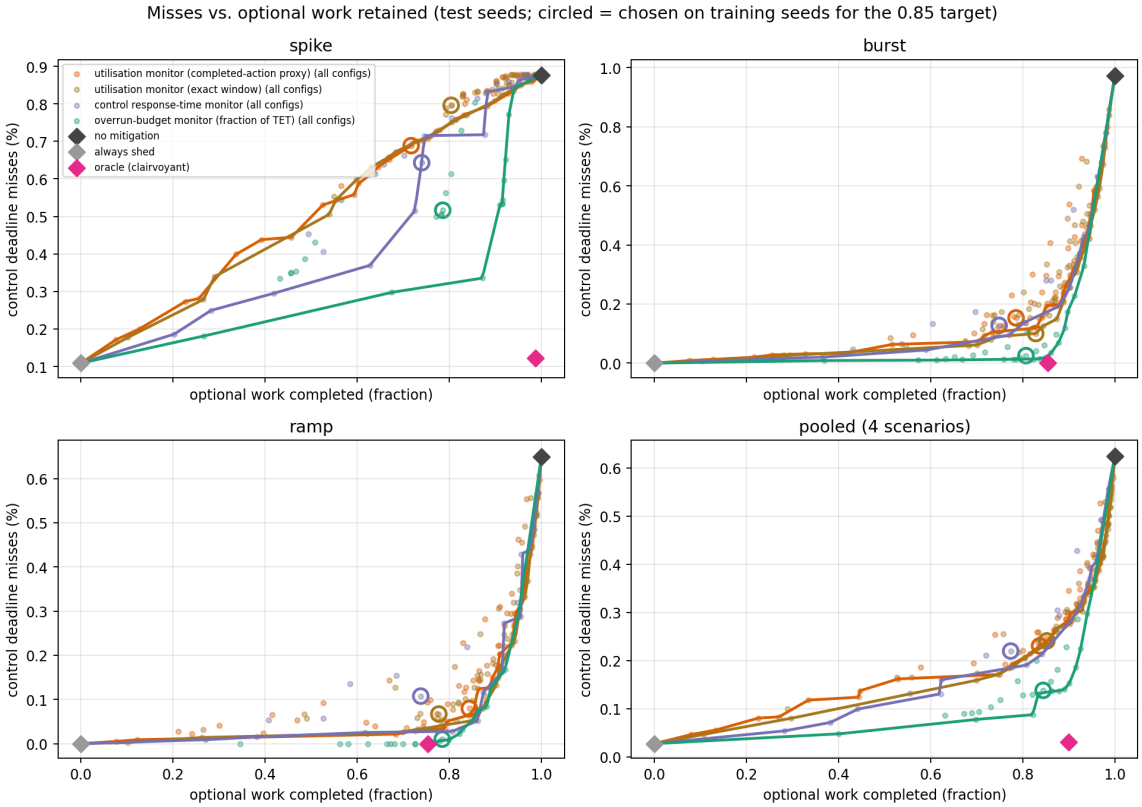


Figure 5: Control deadline misses against retained optional work (test seeds). Dots: every evaluated configuration; lines: exploratory interpolations of empirical Pareto frontiers including the no-mitigation and always-shed reference points (straight segments are interpolations, not measurements of intermediate policies); circles: configurations chosen on training seeds for the 0.85 target; diamonds: reference policies.

Policy	Spike	Burst	Ramp
no mitigation	0.876 (1.00)	0.972 (1.00)	0.649 (1.00)
utilisation proxy	0.689 (0.72)	0.153 (0.79)	0.080 (0.84)
utilisation, exact window	0.796 (0.80)	0.100 (0.83)	0.067 (0.78)
response-time monitor	0.643 (0.74)	0.128 (0.75)	0.108 (0.74)
overrun-budget monitor	0.516 (0.79)	0.025 (0.81)	0.010 (0.78)
oracle (clairvoyant)	0.123 (0.99)	0.000 (0.85)	0.000 (0.75)
always shed	0.109 (0.00)	0.000 (0.00)	0.000 (0.00)

Table 9: Control misses in % (optional work kept in parentheses) per scenario for the configurations selected for 0.85, test seeds. Retention is not matched within a scenario.

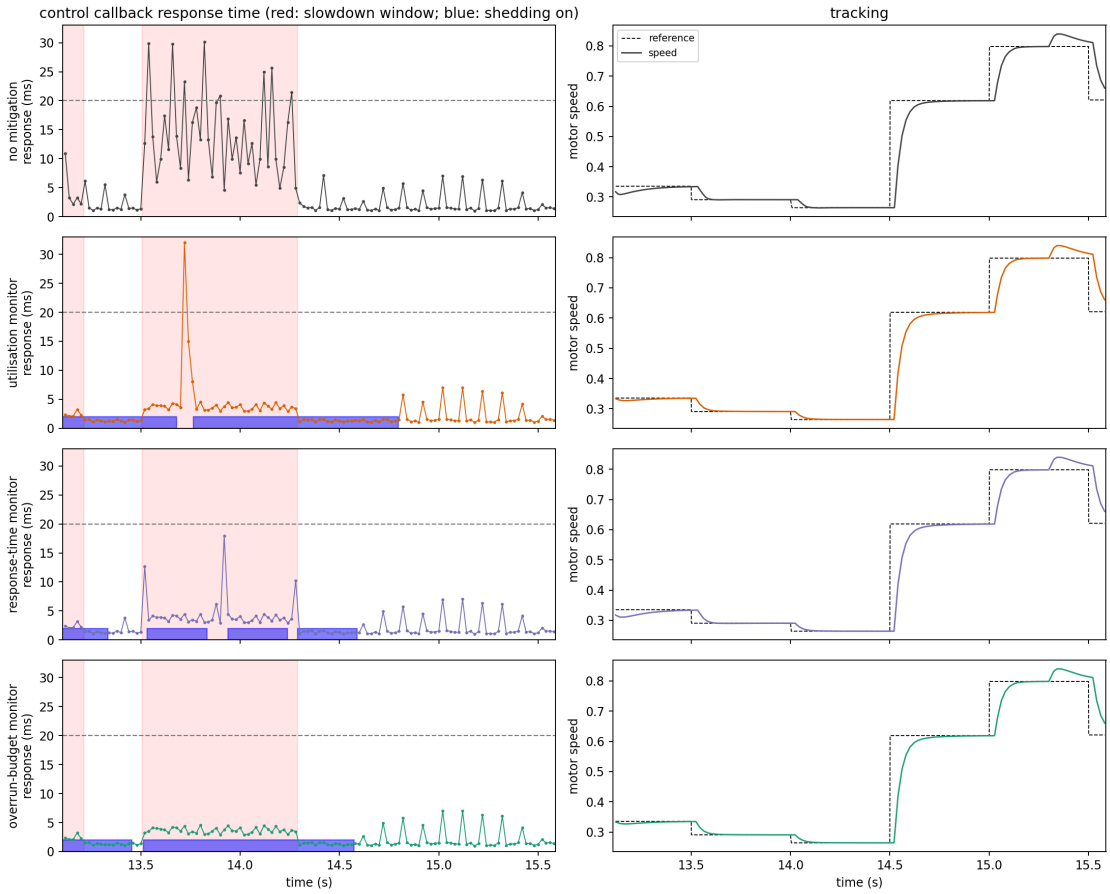


Figure 6: One burst (a single test seed chosen to show many unmitigated misses). Left: control response time (red: slowdown window; blue: shedding active; dashed: deadline). Right: motor speed against its reference. The figure illustrates the mechanism and is not representative of averages.

response time exceeds the deadline repeatedly; the selected monitors shed during most of the window.

What was executing when a control deadline was missed. We classify every missed control job by what was executing when its message arrived (Table 10, Figure 7). Without mitigation, 1887 of the 1973 misses (96%) arrived while an optional job was executing: 622 of 692 in the spike scenario, 757 of 768 in the burst scenario and 508 of 513 in the ramp scenario. This is a temporal association. The classification does not show whether an earlier warning was available or whether the overlap made the miss unavoidable (Observation 1(b) gives a sufficient condition for the latter). With always-shed, all 86 remaining spike-scenario misses arrive during mandatory work. The selected overrun monitor's remaining spike misses include 329 arrivals during an unshed optional job and 77 during mandatory work or a discard stub. Establishing the mechanism of each miss would require additional timing and warning-state analysis.

Cost without disturbance, and coverage of slowdown windows. Table 11 shows the cost of each monitor when only OS noise is present and how often it was shedding during slowdown windows. The utilisation proxy, the exact-window monitor and the selected overrun monitor never

Policy	Spike	Burst	Ramp
no mitigation	622 / 68 / 2	757 / 0 / 11	508 / 0 / 5
utilisation proxy	469 / 72 / 3	119 / 0 / 2	61 / 0 / 2
utilisation, exact window	557 / 69 / 3	79 / 0 / 0	53 / 0 / 0
response-time monitor	440 / 67 / 1	101 / 0 / 0	84 / 0 / 1
overrun-budget monitor	329 / 77 / 2	19 / 0 / 1	8 / 0 / 0
oracle (clairvoyant)	0 / 97 / 0	0 / 0 / 0	0 / 0 / 0
always shed	0 / 86 / 0	0 / 0 / 0	0 / 0 / 0

Table 10: Missed control jobs over all 40 test seeds, classified by the execution interval containing their arrival: an optional job whose body was not shed / necessary work or a discard stub / no other callback executing. The classification does not establish warning history or the cause of a miss.

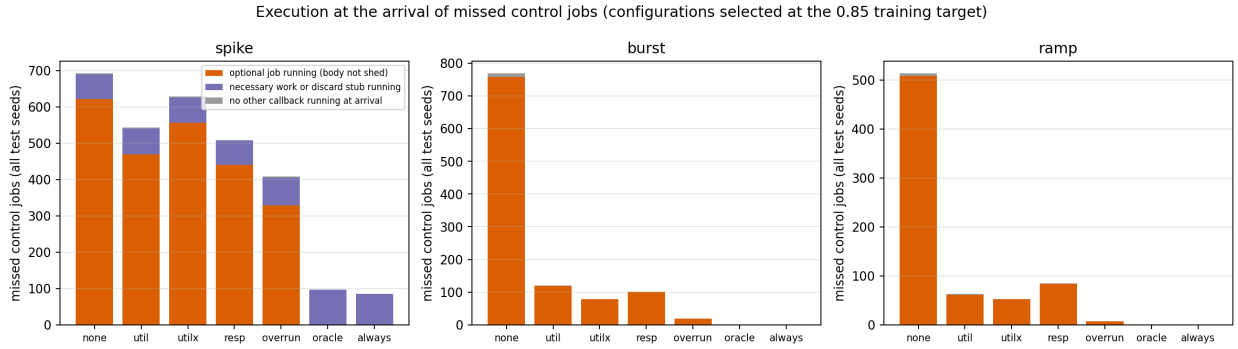


Figure 7: Missed control jobs classified by what was executing at their arrival (all test seeds; configurations selected at the 0.85 training target).

	Nominal scenario		Slowdown windows overlapped by shedding		
Policy	Optional work	False triggers/min	Spike	Burst	Ramp
utilisation proxy	1.000	0.00	0.25	0.78	0.76
utilisation, exact window	1.000	0.00	0.17	0.99	0.98
response-time monitor	0.869	26.96	0.31	0.91	0.88
overrun-budget monitor	1.000	0.00	0.60	0.96	0.98

Table 11: Cost without disturbance and fraction of slowdown windows overlapped by a shedding episode (configurations selected for 0.85, test seeds).

trigger in the nominal scenario and keep all optional work. The selected response-time monitor triggers about 26.96 times per minute there (under the counting convention of Section 4.2) and gives up 13% of the optional work: its 10 ms trigger ($\varphi = 0.5$) is crossed in the nominal simulations, whose analytic response-time bound is 14.0 ms. In slowdown windows the share overlapped by a shedding episode is 0.60, 0.96 and 0.98 for the overrun monitor in the spike, burst and ramp scenarios, against 0.25, 0.78 and 0.76 for the utilisation proxy and 0.17, 0.99 and 0.98 for the exact-window monitor. The windows counted include the mild ones.

6.3 Sizing the overrun budget

Table 12 gives the analysis of two fixed configurations of our workload. Replacing the optional callback bodies by stubs raises the computed TET of `ctrl` from 6.005 ms to 14.005 ms and its

		All callbacks running			Optional work shed		
Callback	D (ms)	$R(0)$	$\hat{e}$ (ms)	$\hat{s}$	$R(0)$	$\hat{e}$ (ms)	$\hat{s}$
`ctrl`	20	13.99	6.00	1.43	6.00	14.01	3.34
`sens`	40	18.08	20.25	2.03	6.09	32.24	5.16
`tele`	50	26.17	20.25	1.47	6.19	36.31	3.65
`viz`	100	26.25	50.58	2.02	6.27	78.55	4.66

Table 12: Analysis of two fixed workload configurations: all callback bodies enabled, and optional bodies replaced by 10 μs stubs. Columns: nominal response-time bound $R(0)$, TET $\hat{e}$ and necessary slowdown $\hat{s}$. The configured scheduler overheads are used without the monitors' extra per-dispatch cost, and transitions between the configurations are not analysed. The lowest-priority recovery bound is sensitive to an arrival-curve discontinuity (Section 5).

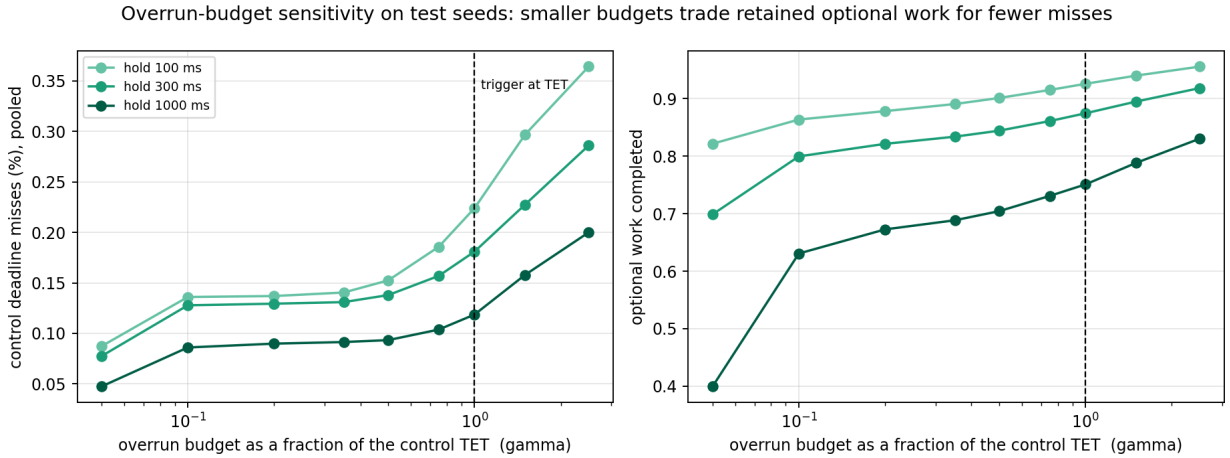


Figure 8: Pooled control misses (left) and retained optional work (right) of the overrun monitor against the budget as a fraction of the control TET, for three hold times (test seeds; $\gamma = 1$ marks triggering at the TET).

necessary-slowdown metric from 1.43 to 3.34. This indicates a larger analysed margin in the steady degraded configuration; it does not establish a guarantee for the monitored system or for its mode transitions.

Figure 8 shows how the pooled results depend on the budget $\gamma\,\hat{e}_{\texttt{ctrl}}$. With $H = 100$ ms the miss rate is 0.087% at $\gamma = 0.05$, between 0.136% and 0.153% for γ from 0.10 to 0.50, 0.224% at $\gamma = 1$ (triggering *at* the TET) and 0.364% at $\gamma = 2.5$, while the retained optional work is 0.822, between 0.864 and 0.901, 0.926 and 0.956. Going from half the TET to the TET itself therefore costs 47% more misses for 2.5 points of optional work. The smallest budget is the cheapest in misses but costs the most optional work (with $H = 1000$ ms it keeps only 0.400), because small budgets are crossed by OS-noise events. The selected thresholds are small fractions of the TET in this workload; because γ is tuned, the experiments do not isolate an advantage of normalising by the TET over tuning an absolute overrun budget, which would require, for example, testing the transfer of a threshold to other task sets.

Policy	Target	Configuration	Train opt.	Test optional work	Test misses (%)
utilisation proxy	0.80	θ=0.80, W=100 ms, H=2000 ms	0.807	0.774 [0.763, 0.786]	0.206 [0.184, 0.230]
utilisation proxy	0.85	θ=0.90, W=30 ms, H=300 ms	0.870	0.854 [0.848, 0.860]	0.249 [0.222, 0.278]
utilisation proxy	0.90	θ=0.50, W=100 ms, H=30 ms	0.913	0.904 [0.901, 0.908]	0.296 [0.262, 0.330]
overrun-budget monitor	0.80	γ=0.05, H=100 ms	0.835	0.822 [0.815, 0.828]	0.087 [0.073, 0.102]
overrun-budget monitor	0.85	γ=0.05, H=50 ms	0.868	0.857 [0.852, 0.863]	0.088 [0.074, 0.103]
overrun-budget monitor	0.90	γ=0.10, H=30 ms	0.907	0.898 [0.893, 0.903]	0.137 [0.114, 0.160]
paired difference overrun − util., target 0.80				0.048 [0.038, 0.058]	-0.119 [-0.136, -0.103]
paired difference overrun − util., target 0.85				0.003 [0.001, 0.006]	-0.161 [-0.183, -0.141]
paired difference overrun − util., target 0.90				-0.007 [-0.011, -0.003]	-0.159 [-0.177, -0.141]

Table 13: Exploratory wider-grid check for the utilisation-proxy and overrun families: configurations re-selected on training seeds over the union of the main and extended grids and evaluated on the original test seeds.

6.4 Robustness

Wider grids (exploratory). Both utilisation and overrun had configurations on the edge of the grid, so we widened them (utilisation windows 20–200 ms, holds 30–2000 ms; overrun γ down to 0.01 and the same holds; 207 additional configurations), re-selected on the training seeds and evaluated the newly selected configurations on the original test seeds (Table 13). At the 0.85 target the wider-grid utilisation choice has 0.249% misses at 0.854 optional work and the wider-grid overrun choice ($\gamma = 0.05$, $H = 50$ ms) 0.088% at 0.857; the paired difference is -0.161 pp (CI -0.183 to -0.141) with $+0.003$ optional work. The grid extension was motivated by inspected results and reuses the test seeds, so it is not validation on a fresh holdout. The utilisation and overrun families were widened; the exact-window and response-time grids were not.

Higher-peak scenarios. Table 14 re-evaluates the selected configurations without retuning on burst and ramp scenarios whose constant or peak slowdown lies between 2× and 3.5× (the ramp still rises from and returns to 1), using the original test-seed range with scenario-specific disturbance streams. Without mitigation the miss rates are 2.19% (burst) and 1.42% (ramp). The selected overrun configuration reaches 0.071% and 0.010% (retained 0.78 and 0.76), the exact-window utilisation monitor 0.209% and 0.118% (0.81 and 0.76), the utilisation proxy 0.367% and 0.141% (0.72 and 0.79), and the response-time monitor 0.315% and 0.180% (0.72 and 0.71).

OS noise. Table 15 multiplies the OS-noise rate by 3 and 10 without retuning. The selected overrun configuration is insensitive: its miss rate goes from 0.138% to 0.139% and 0.146%, its retained optional work from 0.844 to 0.844 and 0.840, and it records 0.0, 0.0 and 0.0 false triggers per minute in the nominal scenario. The more sensitive wider-grid choice ($\gamma = 0.05$, a budget of about 300 μs) has miss rates of 0.088%, 0.085% and 0.076% but triggers 27, 74 and 234 times per minute in the nominal scenario, and its retained optional work falls from 0.857 to 0.819 and 0.673. The utilisation monitors stay at 0.83–0.85 optional work and 0.23–0.25% misses. These results motivate evaluating a threshold against the noise distribution of the target platform and the desired retention; they do not derive a universal threshold or the best budget after retuning at each noise level.

6.5 Control performance

Misses translate only weakly into tracking error at these miss rates: without mitigation the RMS tracking error is 0.78% above the nominal run (pooled). The selected monitor configurations have

Scenario	Policy	Control misses (%)	Optional work
burst, high peak	no mitigation	2.191 [1.895, 2.490]	1.000
burst, high peak	utilisation proxy, θ=1.00, W=50 ms, H=1000 ms	0.367 [0.313, 0.425]	0.721
burst, high peak	utilisation proxy, θ=0.90, W=30 ms, H=300 ms	0.300 [0.246, 0.357]	0.827
burst, high peak	utilisation, exact window, θ=0.50, W=100 ms, H=300 ms	0.209 [0.172, 0.248]	0.808
burst, high peak	response-time monitor, φ=0.50, H=300 ms	0.315 [0.252, 0.386]	0.724
burst, high peak	overrun-budget monitor, γ=0.50, H=300 ms	0.071 [0.048, 0.096]	0.777
burst, high peak	overrun-budget monitor, γ=0.05, H=50 ms	0.054 [0.034, 0.076]	0.818
burst, high peak	oracle (clairvoyant)	0.000 [0.000, 0.000]	0.849
burst, high peak	always shed	0.000 [0.000, 0.000]	0.000
ramp, high peak	no mitigation	1.415 [1.166, 1.673]	1.000
ramp, high peak	utilisation proxy, θ=1.00, W=50 ms, H=1000 ms	0.141 [0.097, 0.189]	0.789
ramp, high peak	utilisation proxy, θ=0.90, W=30 ms, H=300 ms	0.172 [0.119, 0.233]	0.763
ramp, high peak	utilisation, exact window, θ=0.50, W=100 ms, H=300 ms	0.118 [0.082, 0.156]	0.764
ramp, high peak	response-time monitor, φ=0.50, H=300 ms	0.180 [0.127, 0.239]	0.706
ramp, high peak	overrun-budget monitor, γ=0.50, H=300 ms	0.010 [0.004, 0.019]	0.758
ramp, high peak	overrun-budget monitor, γ=0.05, H=50 ms	0.000 [0.000, 0.000]	0.708
ramp, high peak	oracle (clairvoyant)	0.000 [0.000, 0.000]	0.741
ramp, high peak	always shed	0.000 [0.000, 0.000]	0.000

Table 14: Configurations selected on the main scenarios, without retuning, on higher-peak burst and ramp scenarios (original test-seed range). The ramp keeps its triangular profile.

Policy (not re-tuned)	noise ×1	noise ×3	noise ×10
utilisation proxy, θ=1.00, W=50 ms, H=1000 ms	0.230 / 0.837 / 0.0	0.233 / 0.835 / 0.0	0.241 / 0.830 / 0.0
utilisation proxy, θ=0.90, W=30 ms, H=300 ms	0.249 / 0.854 / 0.0	0.249 / 0.853 / 0.1	0.259 / 0.847 / 1.2
utilisation, exact window, θ=0.50, W=100 ms, H=300 ms	0.241 / 0.852 / 0.0	0.244 / 0.851 / 0.0	0.249 / 0.848 / 0.0
response-time monitor, φ=0.50, H=300 ms	0.220 / 0.774 / 27.0	0.219 / 0.771 / 27.5	0.226 / 0.758 / 30.2
overrun-budget monitor, γ=0.50, H=300 ms (selected)	0.138 / 0.844 / 0.0	0.139 / 0.844 / 0.0	0.146 / 0.840 / 0.0
overrun-budget monitor, γ=0.05, H=50 ms (wider grid)	0.088 / 0.857 / 27.1	0.085 / 0.819 / 73.9	0.076 / 0.673 / 233.7
no mitigation	0.624 / 1.000 / 0.0	0.641 / 1.000 / 0.0	0.677 / 1.000 / 0.0

Table 15: Pooled control misses (%) / retained optional work / false triggers per minute in the nominal scenario, for OS-noise rates multiplied by 1, 3 and 10 (test seeds, configurations not retuned).

small mean RMS excess values between −0.23% and 0.11%. The marginal intervals overlap, but this alone is not a paired test, and the study does not establish a consistent control-quality advantage for one monitor. Shedding everything lowers the tracking error by about 2% even in the nominal scenario, because optional callbacks otherwise delay the control job. The mean longest run of consecutive misses (Table 6) falls from 1.21 without mitigation to 1.04, 1.04 and 1.04 for the baselines and 0.69 for the overrun monitor.

7 Discussion

7.1 Interpretation

The overrun monitor's advantage is consistent with three properties of the model and of the signal, none of which we isolated by an ablation. First, the model applies a common time-dependent slowdown to all actions, so short completed actions (the control callback, the scheduler's own actions) can show an overrun before a long optional callback is dispatched; this is a plausible explanation for early shedding under correlated disturbances. Second, the overrun signal uses per-action NETs, whereas the utilisation monitors aggregate executed durations and the response-time monitor

k	`viz` / `tele` chunk (ms)	$R_{\texttt{ctrl}}(0)$ (ms)	$\hat{e}_{\texttt{ctrl}}$ (ms)	Algebraic $s^\star$ for an optional chunk
1	12.00 / 8.00	13.99	6.00	1.50
2	6.00 / 4.00	8.16	11.84	2.95
3	4.00 / 2.67	6.32	13.60	4.33
4	3.00 / 2.00	6.48	13.44	5.35
6	2.00 / 1.33	6.80	12.95	7.21
12	1.00 / 0.67	7.86	11.74	11.81

Table 16: Analytic what-if: optional callbacks represented by shorter costs and a burst-permitting arrival curve (same total work), rounded to integer microseconds. The last column is the algebraic factor $1+\hat{e}_{\texttt{ctrl}}/C_{\text{ovh}}$ for an optional chunk, not an unavoidable-miss threshold. Analysed, not simulated.

observes completed control jobs; the selected response-time configuration triggers even when only OS noise is present (Table 11). These observations are consistent with different sensitivities to workload and disturbances but do not separate NET-relative thresholds from other implementation and tuning differences. Third, the selected causal monitors retain more misses in the spike scenario than the oracle (Table 9); this is consistent with the difficulty of reacting after a nonpreemptive callback has started, but neither the scenario comparison nor Observation 1 shows that all residual misses are unavoidable or that every causal monitor must behave like those evaluated. An ablation that separates NET-relativity from per-action granularity is a natural follow-up.

7.2 Practical guidance

(i) Measure action costs and the disturbance distribution on the target platform, and judge a threshold by its miss rate *and* its retained optional work; the smallest budgets tested here become costly as the OS-noise rate rises (Table 15). (ii) Use the TET as a reported normalisation, not as a proved trigger rule: the budget sweep shows a trade-off, with misses lowest at the smallest budgets and rising steeply beyond half the TET (Figure 8). (iii) Holds of 30–300 ms were selected at the 0.85 target in the tested grids; at fixed γ the longer holds generally reduce misses while discarding more optional work (Table 17). (iv) Reducing the length of nonpreemptive callbacks is a promising direction suggested by the fixed-configuration analysis below, but its runtime benefit and cost require evaluation.

7.3 An analytic what-if: shorter optional callbacks

Table 16 explores an abstract workload change: each optional callback is assigned cost NET$/k$, period T/k and release jitter $(k-1)T/k$ (rounded to integer microseconds), so that the arrival curve permits a burst of up to k chunks and the long-run demand is approximately unchanged. We have not established that this arrival model captures a particular continuation mechanism. At $k = 3$ the computed TET of `ctrl` rises from 6.005 ms to 13.60 ms and the algebraic factor $1 + \hat{e}_{\texttt{ctrl}}/C_{\text{ovh}}$ for the largest optional chunk from about 1.50 to 4.33. Among the evaluated values of k the TET is largest at $k = 3$; further splitting increases the modelled overhead and lowers it again. The what-if is analytic only: it ignores the cost of posting continuations and has not been simulated, and we do not claim that the monitors would perform as in Section 6 on a chunked workload.

7.4 Threats to validity

Internal. Unit tests, the comparison with the published workload and finite adversarial runs are implementation checks, not a proof of simulator correctness or of correspondence with the verified C

scheduler. They do not exhaust the behaviour of the monitors. The simulator skips idle iterations; with OS-noise events replayed inside the skipped intervals the skip changed the retained optional work of the overrun monitor by 0.0015 on average in a sample of 96 paired runs (Section 5), but equivalence with a fully action-by-action simulation is not proved. *Construct.* Pooled metrics give the four scenarios equal weight and optional work is NET-weighted. The overrun signal is a per-action heuristic with an idle-reset rule; the utilisation proxy is a completed-action approximation, which the exact-window monitor addresses. The miss classification records execution at arrival, not causality or prior warnings. The false-trigger counts use a different observation window from the job metrics. The oracle and always-shed are diagnostic references, not optimal bounds. *External.* One core, one control loop, illustrative overheads, synthetic and spatially uniform slowdowns, and a first-order plant; real contention may be less correlated across jobs, which would weaken the precursor effect, and real noise may be heavier-tailed. Disturbance rates were set so that the baseline misses about 1% of its deadlines. *Statistical.* The main comparison uses 40 test seeds. Selecting configurations on training seeds separates selection from evaluation but does not guarantee equal retention on the test seeds. The frontiers are built from test results, and the response-time baseline, the exact-window baseline and the wider-grid study were added after results were inspected and reuse the test seeds; we treat them as exploratory. No correction for multiple comparisons was applied, and no equivalence or paired superiority test for tracking error is reported. *Formal.* No guarantee is claimed for the mitigated system.

8 Related Work

Execution-time estimates and exceedance. Static and measurement-based timing analysis, and their limits on modern hardware, are surveyed in [22, 27]; probabilistic schedulability analysis in [15]. Sensitivity analysis asks how much execution times may grow before a fixed-priority system becomes unschedulable [7]; the effect of rare timing events on control quality is studied in [19]; correlated execution times are handled without distributional assumptions in [23], and task models can be inferred from traces of shared-thread code [28]. Exceedance analysis [29] bounds the effect of transient overload on response times, and [5] mechanises it for the Rössl scheduler on top of the PROSA ecosystem [4, 9, 13]. This report builds directly on the latter and evaluates the use of its thresholds that its authors leave to future work.

Nonpreemptive fixed-priority scheduling and ROS 2 executors. Response-time analysis for nonpreemptive fixed-priority scheduling with blocking goes back to [17] and its correction [16]; the blocking term $B(\tau)$ above is of this kind. Response-time analyses of ROS 2 executors and processing chains appear in [8, 12], and starvation in the multithreaded executor is addressed in [25].

Overload management and mixed criticality. Separating necessary from optional work is the idea of imprecise computation [20]; skipping instances [18] and adapting rates [11] are alternative responses to overload; bandwidth-isolating servers confine an overrun to its own budget [1] but rely on preemption. Mixed-criticality scheduling [10, 26] formalises criticality-dependent execution-time assurance, and adaptive mixed-criticality (AMC) analysis [3] switches to a high-criticality mode, discarding low-criticality tasks, when a job exceeds its low-criticality budget; automotive operating systems offer comparable execution-budget supervision [2]. Overrun-triggered degradation is therefore established. This study evaluates one heuristic based on accumulated per-action overrun in a simulation of a nonpreemptive polling executor, expresses its thresholds relative to an analysis-derived TET, and compares it with utilisation and symptom-based triggers under synthetic

disturbances; it also separates the loss of the analysis's sufficient deadline certificate from a miss that is already forced by nonpreemptive blocking (Observation 1). The experiments do not establish a general advantage of normalising by the TET, nor a universal limit for runtime monitors. The mixed-criticality analyses provide guarantees for the mode change that we do not.

Runtime monitoring and control under timing faults. PAStime [24] is a runtime progress-aware scheduling approach for time-critical computing; we regard it as complementary and have not compared against it. The effect of timing on control performance is analysed in [14]; weakly-hard models bound how many deadlines may be missed [6], and stability under consecutive misses is analysed in [21]; combining such control-side tolerance with the thresholds studied here is an open direction. Our literature search was not systematic and we may have missed relevant work.

9 Conclusion and Future Work

We evaluated heuristic overrun-triggered shedding in a simulation of a nonpreemptive polling executor. At the 0.85 training target the selected overrun monitor had 0.138% control deadline misses and retained 0.844 of the optional work, compared with 0.230% and 0.837 for the utilisation proxy, 0.241% and 0.852 for the exact-window utilisation monitor, and 0.220% and 0.774 for the response-time monitor. These results support an advantage of the selected overrun configuration in this workload, with the stated differences in retention; the test-derived frontiers (miss-rate ratios between 0.42 and 0.60) and the supplementary studies are exploratory. Smaller thresholds trade retained optional work for fewer misses, and the retention cost of the most sensitive thresholds grows with the OS-noise rate. The TET is a sufficient-analysis budget, not an unavoidable-miss threshold or a proved runtime trigger. Although 96% of the unmitigated missed control jobs arrived during optional execution, the classification does not establish that those misses were unavoidable.

The evidence is limited to a simulation with illustrative parameters. The most important next steps are: (1) run the executor with measured NETs and injected disturbances on real hardware and compare the simulator with it; (2) analyse the mode change itself, to bound the misses under the monitor and not only in the steady degraded mode; (3) repeat the study on the workload of [5] and on traces of real ROS 2 applications; (4) add AMC-style and reservation-based baselines; (5) implement and evaluate chunked optional callbacks, for which Table 16 predicts a larger sufficient-analysis margin under the stated abstract arrival model; and (6) run an ablation separating NET-relativity from per-action granularity.

Reproducibility

The accompanying repository (`https://github.com/mohamedghoneim281/overrun-monitor`) contains the analysis, the simulator, the monitor implementations, the stored per-run results and 16 test functions. The main pipeline is run with `bash setup_and_run.sh full`; the wider-grid study and the idle-skip check are separate stages (Appendix 12). The final verification script checks selected numerical relationships and agreement with the stored outputs; it does not validate every scientific claim or prove the implementation correct. All randomness is seeded. The full pipeline was run on Linux x86-64 (Python 3.12) and, independently, on macOS arm64 (Apple M1, Python 3.14). The raw per-run results of the two runs agree to 10^{-16}, and all verification checks passed on both machines. The utilisation-family selections use a rule that ranks miss rates rounded to 10^{-12}, so that exact ties are not decided by floating-point rounding; both machines select the same configurations.

10 Budget sensitivity table

γ	H=100 ms	H=300 ms	H=1000 ms
0.05	0.087 / 0.822	0.078 / 0.699	0.047 / 0.400
0.10	0.136 / 0.864	0.128 / 0.800	0.086 / 0.631
0.20	0.137 / 0.878	0.129 / 0.821	0.090 / 0.673
0.35	0.141 / 0.891	0.131 / 0.834	0.091 / 0.689
0.50	0.153 / 0.901	0.138 / 0.844	0.093 / 0.704
0.75	0.186 / 0.915	0.157 / 0.861	0.104 / 0.731
1.00	0.224 / 0.926	0.181 / 0.875	0.119 / 0.751
1.50	0.297 / 0.940	0.228 / 0.895	0.158 / 0.788
2.50	0.364 / 0.956	0.286 / 0.918	0.200 / 0.830

Table 17: Pooled control misses (%) / retained optional work of the overrun monitor, by budget fraction γ (of $\hat{e}_{\texttt{ctrl}} = 6.005\,\text{ms}$) and hold time H (test seeds).

11 Overrun monitor in pseudo-code

Algorithm 1 Overrun-budget monitor: hooks called by the executor.

state: $\varepsilon \leftarrow 0$ (accumulated per-action overrun since the last idle reset), $t_{\text{shed}} \leftarrow -\infty$
procedure OnActionEnd(action a, time t, duration d_a)
  if $d_a > \text{NET}_a$ **then**
    $\varepsilon \leftarrow \varepsilon + (d_a - \text{NET}_a)$
    if $\varepsilon \geq \gamma\,\hat{e}_{\texttt{ctrl}}$ **then**
      $t_{\text{shed}} \leftarrow \max(t_{\text{shed}}, t + H)$ ▷ (re)start the hold period
    end if
  end if
end procedure
procedure OnQuietInstant ▷ implementation hook after idling; not a proved formal quiet instant
  $\varepsilon \leftarrow 0$
end procedure
function ShedNow(time t) ▷ asked once per optional job, before its dispatch
  return $t < t_{\text{shed}}$
end function

12 How to reproduce

1. Run `bash setup_and_run.sh smoke` as a pipeline check, or `bash setup_and_run.sh full` for the main experiment (including the baselines). The setup script recreates its virtual environment and output directories; use a separate checkout, or preserve any results you need, before rerunning it. Runtime depends on the machine and the package environment.
2. After a full run, activate its environment with `source .venv/bin/activate`, then run `python run_all.py --mode full --stage extend` followed by `python run_all.py --mode full --stage analyze`. The robustness stage simulates only the policies missing from its cache.

3. The idle-skip check of Section 5: `python run_all.py --mode full --stage idlecheck`. The controller calibration of Section 3: `python calibration_latency.py`. The what-if of Table 16: `python chunking_whatif.py`, which applies the analysis of `ovb.analysis` with the optional callbacks' NET, period and release jitter replaced by round(NET/k), round(T/k) and round($(k-1)T/k$) and the original scheduler overheads.

References

[1] Luca Abeni and Giorgio Buttazzo. Integrating multimedia applications in hard real-time systems. In *Proc. 19th IEEE Real-Time Systems Symposium (RTSS)*, pages 4–13, 1998.

[2] *Specification of Operating System, AUTOSAR Classic Platform*. AUTOSAR, n.d.

[3] Sanjoy K. Baruah, Alan Burns, and Robert I. Davis. Response-time analysis for mixed criticality systems. In *Proc. 32nd IEEE Real-Time Systems Symposium (RTSS)*, pages 34–43, 2011.

[4] Kimaya Bedarkar, Lennard Elbeheiry, Michael Sammler, Lennard Gäher, Björn B. Brandenburg, Derek Dreyer, and Deepak Garg. RefinedProsa: Connecting response-time analysis with C verification for interrupt-free schedulers. *Proc. ACM on Programming Languages (PLDI)*, 9: 73–97, 2025.

[5] Kimaya Bedarkar, Bite Ye, Deepak Garg, Derek Dreyer, and Björn B. Brandenburg. From estimates to guarantees: Verified exceedance analysis on commodity hardware. In *Proc. IEEE Real-Time Systems Symposium (RTSS)*, 2026. to appear; preprint.

[6] Guillem Bernat, Alan Burns, and Albert Llamosí. Weakly hard real-time systems. *IEEE Transactions on Computers*, 50(4):308–321, 2001.

[7] Enrico Bini, Marco Di Natale, and Giorgio C. Buttazzo. Sensitivity analysis for fixed-priority real-time systems. *Real-Time Systems*, 39(1–3):5–30, 2008.

[8] Tobias Blaß, Daniel Casini, Sergey Bozhko, and Björn B. Brandenburg. A ROS 2 response-time analysis exploiting starvation freedom and execution-time variance. In *Proc. 42nd IEEE Real-Time Systems Symposium (RTSS)*, 2021.

[9] Sergey Bozhko and Björn B. Brandenburg. Abstract response-time analysis: A formal foundation for the busy-window principle. In *Proc. 32nd Euromicro Conf. on Real-Time Systems (ECRTS)*, pages 22:1–22:24, 2020.

[10] Alan Burns and Robert I. Davis. A survey of research into mixed criticality systems. *ACM Computing Surveys*, 50(6):82:1–82:37, 2017.

[11] Giorgio C. Buttazzo, Giuseppe Lipari, and Luca Abeni. Elastic task model for adaptive rate control. In *Proc. 19th IEEE Real-Time Systems Symposium (RTSS)*, pages 286–295, 1998.

[12] Daniel Casini, Tobias Blaß, Ingo Lütkebohle, and Björn B. Brandenburg. Response-time analysis of ROS 2 processing chains under reservation-based scheduling. In *Proc. 31st Euromicro Conf. on Real-Time Systems (ECRTS)*, volume 133 of *LIPIcs*, 2019.

[13] Felipe Cerqueira, Felix Stutz, and Björn B. Brandenburg. PROSA: A case for readable mechanized schedulability analysis. In *Proc. 28th Euromicro Conf. on Real-Time Systems (ECRTS)*, pages 273–284, 2016.

[14] Anton Cervin, Dan Henriksson, Bo Lincoln, Johan Eker, and Karl-Erik Årzén. How does control timing affect performance? analysis and simulation of timing using Jitterbug and TrueTime. *IEEE Control Systems Magazine*, 23(3):16–30, 2003.

[15] Robert I. Davis and Liliana Cucu-Grosjean. A survey of probabilistic schedulability analysis techniques for real-time systems. *Leibniz Trans. Embedded Systems*, 6(1):04:1–04:53, 2019.

[16] Robert I. Davis, Alan Burns, Reinder J. Bril, and Johan J. Lukkien. Controller area network (CAN) schedulability analysis: Refuted, revisited and revised. *Real-Time Systems*, 35(3): 239–272, 2007.

[17] Laurent George, Nicolas Rivierre, and Marco Spuri. Preemptive and non-preemptive real-time uniprocessor scheduling. Technical Report RR-2966, INRIA, 1996.

[18] Gilad Koren and Dennis Shasha. Skip-over: Algorithms and complexity for overloaded systems that allow skips. In *Proc. 16th IEEE Real-Time Systems Symposium (RTSS)*, pages 110–117, 1995.

[19] Pratyush Kumar and Lothar Thiele. Quantifying the effect of rare timing events with settling-time and overshoot. In *Proc. 33rd IEEE Real-Time Systems Symposium (RTSS)*, pages 149–160, 2012.

[20] Jane W. S. Liu, Kwei-Jay Lin, Wei-Kuan Shih, Albert Chuang shi Yu, Jen-Yao Chung, and Wei Zhao. Algorithms for scheduling imprecise computations. *IEEE Computer*, 24(5):58–68, 1991.

[21] Martina Maggio, Arne Hamann, Eckart Mayer-John, and Dirk Ziegenbein. Control-system stability under consecutive deadline misses constraints. In *Proc. 32nd Euromicro Conf. on Real-Time Systems (ECRTS)*, volume 165 of *LIPIcs*, 2020.

[22] Claire Maiza, Hamza Rihani, Juan M. Rivas, Joel Goossens, Sebastian Altmeyer, and Robert I. Davis. A survey of timing verification techniques for multi-core real-time systems. *ACM Computing Surveys*, 52(3):56:1–56:38, 2019.

[23] Filip Marković, Georg von der Brüggen, Mario Günzel, Jian-Jia Chen, and Björn B. Brandenburg. A distribution-agnostic and correlation-aware analysis of periodic tasks. In *Proc. 45th IEEE Real-Time Systems Symposium (RTSS)*, pages 215–228, 2024.

[24] Soham Sinha, Richard West, and Ahmad Golchin. PAStime: Progress-aware scheduling for time-critical computing. In *Proc. 32nd Euromicro Conf. on Real-Time Systems (ECRTS)*, volume 165 of *LIPIcs*, pages 3:1–3:24, 2020. doi: 10.4230/LIPIcs.ECRTS.2020.3.

[25] Harun Teper, Daniel Kuhse, Mario Günzel, Georg von der Brüggen, Falk Howar, and Jian-Jia Chen. Thread carefully: Preventing starvation in the ROS 2 multithreaded executor. *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 43(11):3588–3599, 2024.

[26] Steve Vestal. Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance. In *Proc. 28th IEEE Real-Time Systems Symposium (RTSS)*, pages 239–243, 2007.

[27] Reinhard Wilhelm, Jakob Engblom, Andreas Ermedahl, Niklas Holsti, Stephan Thesing, David Whalley, Guillem Bernat, Christian Ferdinand, Reinhold Heckmann, Tulika Mitra, Frank Mueller, Isabelle Puaut, Peter Puschner, Jan Staschulat, and Per Stenström. The worst-case

execution-time problem—overview of methods and survey of tools. *ACM Trans. Embedded Computing Systems*, 7(3):36:1–36:53, 2008.

[28] Bite Ye, Filip Marković, and Björn B. Brandenburg. Framework-agnostic model inference for intra-thread real-time tasks. In *Proc. 32nd IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, pages 68–81, 2026.

[29] Matteo Zini, Filip Marković, Daniel Casini, Alessandro Biondi, and Björn B. Brandenburg. In search of butterflies: Exceedance analysis for real-time systems under transient overload. In *Proc. 45th IEEE Real-Time Systems Symposium (RTSS)*, pages 229–242, 2024.